

Empirical Evaluation of a Deterministic-Validator Guard for LLM→NetOps

Generate→Verify→Repair Pipelines (Revised)

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment that measures the operational impact of inserting a deterministic network-configuration validator into an LLM-driven generate→verify→repair (GVR) loop for intent→configuration NetOps tasks. Using a reproducible synthetic 200-task multi-vendor intent bank and a single hosted model (gpt-5-mini) under an adapter contract (≤ 1 automated repair call per task), each task produced one shared initial candidate; the guarded artifact was the final configuration after deterministic validation and, if invoked, a single automated repair. Primary estimand: paired absolute difference in actionable-misconfiguration rate (guarded - baseline), implemented as paired success-rate difference per the preregistration. Results (N=200 pairs): baseline valid-config count = 199/200 (0.995), guarded valid-config count = 200/200 (1.000); absolute paired change (guarded - baseline) = 0.005 (0.5 percentage points). Exact McNemar two-sided test $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.015] (10,000 resamples, seed 1729). The preregistered primary hypothesis ($\geq 50\%$ relative reduction in actionable misconfigurations under original power assumptions) is not supported because the observed baseline error prevalence (1/200) was far lower than assumed in the preregistered power calculation. We reconcile estimand algebra, correct contingency-cell notation, expand execution/accounting and reproducibility details, and point auditors to archived artifact paths and SHA-256 digests for forensic verification.

I. INTRODUCTION

Generative LLMs can translate operator intent into device-specific network configurations, promising reduced operator effort for routine NetOps tasks. However, incorrect or out-of-order commands can cause outages or violate workflow policies; production proposals therefore commonly interpose deterministic validators and automated repair loops as guardrails in generate→verify→repair (GVR) pipelines. Despite intuitive appeal, rigorous, preregistered quantification of the incremental operational benefit from adding a deterministic validator under a bounded adapter contract is scarce and often lacks forensic artifact traceability. We preregistered study `cnsmspec2026_gvr_v1_prereg_001` (SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099cb7da6ac4) to answer: How much does integrating a deterministic network validator into an LLM-driven GVR pipeline reduce actionable misconfiguration rates (paired comparison) on a 200-task synthetic multi-vendor NetOps task bank, and what residual error types persist after automated repairs? Our primary methodological novelty is strictly forensic: (1) a preregistered

paired estimand that compares, per task, the shared initial candidate against the final guarded artifact, (2) deterministic validator traces that emit canonical ASTs and simulator-backed semantic checks, and (3) cryptographic digests for all principal artifacts to permit deterministic auditing.

II. RELATED WORK

Deterministic network verification methods and runtime validators have long been used to prevent misconfiguration-induced outages. Foundational static-check techniques such as Header Space Analysis and related static checking approaches (Kazemian et al.; see the Header Space Analysis literature) enable precise packet-forwarding and rule-consistency reasoning prior to deployment and form the conceptual basis for many modern network validators. Incremental and online checkers (e.g., VeriFlow [Khurshid et al., DOI:10.1145/2342441.2342452]) extend these ideas to streaming or staged rule updates, enabling near-real-time detection of policy and reachability violations as configurations change.

Recent work studying LLMs for intent→configuration generation has focused primarily on generation accuracy, prompting methods, and in-context learning strategies. NetConfEval and related evaluations (e.g., NetConfEval: Can LLMs Facilitate Network Configuration? [Wang et al., DOI:10.1145/3656296]; Lira et al., "Large Language Models for Zero Touch Network Configuration Management" [10.1109/mcom.001.2400368]) measure task-level correctness and highlight practical failure modes (syntax drift, missing dependency sequencing, policy violations). Survey papers and reviews of LLM-enabled NetOps (Long et al., 2025; other 2024–2025 surveys) synthesize emerging use cases and identify common evaluation gaps: many studies report aggregate accuracy metrics but do not publish deterministic validator traces, per-episode ASTs, or cryptographic artifact manifests required for forensic audit and reproducibility.

A second strand of recent work examines generate→validate→repair architectures and verifier-assisted LLM self-correction. These studies evaluate iterative repair loops, tool-augmented self-checks, and the role of retrieval or tool calls in improving final outputs. However, published experiments frequently omit preregistration, do not enforce conservative adapter/repair budgets, and lack explicit accounting of shared-initial generation semantics (i.e., whether baseline and guarded conditions share

an identical initial candidate). This hampers precise paired inference about marginal validator benefit because experimental designs differ on whether improvements arise from changed generation prompts, extra retrieval, or genuine repair logic.

Our study situates itself at the intersection of these literatures. We employ deterministic-validator ideas from the verification literature (header-space and incremental checking) combined with LLM-driven generation and a constrained GVR adapter that enforces: (a) a single shared_initial generation reused for both baseline and guarded conditions, and (b) an explicitly limited repair budget (≤ 1 automated repair call per task). Unlike many prior LLM $\rightarrow$ NetOps evaluations, our contribution emphasizes preregistration, paired estimand clarity, and artifact-forensics (per-episode validator_trace objects, canonical ASTs, and SHA-256 digests for all principal files) so that auditors can deterministically reconstruct contingency cells and inspect the exact discordant episodes. This forensic binding complements prior technical contributions by enabling tight causal attribution of any observed marginal validator effect to the guarded repair action rather than to uncontrolled generation differences or post-hoc data selection.

III. METHODOLOGY

Preregistration and estimand. We preregistered a paired binary design with $N=200$ intent $\rightarrow$ configuration tasks stratified by planned vendor family (planned strata: Cisco-like $N=66$; Juniper-like $N=67$; RFC-neutral $N=67$) and defined the primary_estimand as paired_success_rate_difference_guarded_minus_baseline (success = non-actionable configuration). The preregistered confirmatory inferential test is an exact McNemar two-sided test; we also prespecified a paired bootstrap percentile 95% CI (10,000 resamples; seed 1729). Full preregistration (cnsmspec2026_gvr_v1_prereg_001) SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099eb716ac4.

Adapter contract and generation semantics. Execution used adapter_family = hosted_netops_gvr_v1 and requested_model = gpt-5-mini (execution/model_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd0733578051820). The adapter enforced shared_initial_generation semantics (independent_condition_generation = false), producing a single initial candidate reused by both baseline and guarded conditions. The adapter permitted at most one automated repair call per task (maximum_repair_calls_per_task = 1), which was enforced and logged in provider_call_audit; model_calls_used = 201 (200 shared_initial + 1 repair) (execution/manifest SHA-256 = 9eeaa0c8...; deterministic_reconciliation.json SHA-256 = 8a2b85f8...).

Synthetic task bank, validator, and scoring. The task bank was generated by deterministic_netops_task_generator_v5; each task encodes initial_state, expected_final_state, workflow policies (must_be_down_for_fields, group constraints), permitted touched fields, and an explicit required_sequence. The deterministic validator (deterministic_netops_validator_v5) parses model outputs into canonical ASTs, enforces vendor syntax/parse, applies intent-constraint and dependency checks,

and runs simulator-backed semantic assertions; the validator stores per-episode validator_trace objects (validation_before, validation_after) and returns a binary score using scoring_rule = full_intent_constraint_validation. Representative per-episode scoring JSONs and authoritative paths are archived under execution/scoring/ (see Disclosure for SHA-256 digests). Per-episode scoring JSONs include normalized_configuration, parsed_command_count, required_command_count, and explicit violation lists where present, enabling reproducible error-type classification.

Execution, retries, and missingness rules. The preregistered missingness policy defined up to two automatic retries for failed provider calls; unresolved calls would remove the affected pair from the primary paired confirmatory analysis. No provider-call failures occurred. Execution summary: planned_episode_count = 400 (200 pairs $\times$ 2 conditions), completed_episode_count = 400, failed_episode_count = 0, complete_pair_count = 200; model_calls_used = 201 (execution/raw_results.jsonl SHA-256 = 9eeaa0c8...). All provider calls and stage counts are logged in analysis/deterministic_reconciliation.json (SHA-256 = 8a2b85f8...).

Statistical analysis pipeline. The analysis executor paired_binary_analysis_v1 consumed execution/raw_results.jsonl and produced the contingency table (n_11,n_10,n_01,n_00), exact McNemar two-sided p-value, and paired bootstrap percentile CI for the absolute paired difference (bootstrap_seed = 1729; resamples = 10,000). All analysis artifacts (analysis/paired_contingency_table.csv SHA-256 = 658051ba..., analysis/condition_summary.csv SHA-256 = 86ab6b56...) are archived. We preserved the preregistered multiplicity plan (one primary confirmatory hypothesis; secondary/exploratory tests labeled as exploratory).

IV. RESULTS

Primary confirmatory outcome (artifact-grounded). Complete_pair_count = 200. Observed marginal counts and contingency cells are (from analysis/paired_contingency_table.csv SHA-256 = 658051820).

	Guarded	Baseline
n_11	199	1
n_10	1	0
n_01	0	1
n_00	0	0

 These yield baseline successes = 199/200 (baseline_success_rate = 0.995) and guarded successes = 200/200 (guarded_success_rate = 1.000). The absolute paired difference (guarded - baseline) = 0.005 (0.5 percentage points). Exact McNemar two-sided test (pre-registered) gives $p = 1.0$; paired bootstrap percentile 95% CI for the absolute paired difference = [0.0, 0.015] (10,000 resamples; seed 1729). All numbers and test outputs are archived (analysis/condition_summary.csv SHA-256 = 86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed00628a68... analysis/analysis_log.jsonl SHA-256 = dbc0bd6b...).

Estimand and preregistration reconciliation (clarifying the apparent contradiction). The preregistration text described H1 as a $\geq 50\%$ relative reduction in the actionable-error rate, while the archived primary_estimand and confirmatory

test implemented the paired absolute success-rate difference (paired_success_rate_difference_guarded_minus_baseline).

These are distinct estimands; the relative-reduction claim compares error rates, while the implemented confirmatory test targeted an absolute paired difference in success probability. Algebraic mapping using observed counts clarifies consequences: define $\text{baseline_error} = 1 - \text{baseline_success}$. With observed $\text{baseline_success} = 0.995$, $\text{baseline_error} = 0.005$. Observed $\text{guarded_error} = 1 - \text{guarded_success} = 0.0$. The observed relative reduction $= (\text{baseline_error} - \text{guarded_error}) / \text{baseline_error} = (0.005 - 0) / 0.005 = 1.0$ (100% relative reduction). If one interprets H1 in its literal preregistered wording ($\geq 50\%$ relative reduction), the observed data satisfy that inequality ($100\% \geq 50\%$). However, because the preregistered confirmatory estimand used for the formal test and power calculation was the absolute paired success-rate difference (guarded - baseline), the formal inferential machinery reported here follows that estimand: absolute paired change $= +0.005$ with two-sided exact McNemar $p = 1.0$ and bootstrap 95% CI $= [0.0, 0.015]$. Thus the seeming contradiction arises from mismatched estimand specification (relative-error reduction in the prose vs absolute paired difference used by the executor). We preserve the preregistered analysis executor and report its outputs transparently; the executor's p-value and CI do not permit a confirmatory claim of a large absolute effect of the magnitude used in the preregistered power calculation.

Statistical interpretation and rare-failure regime nuance. The formal confirmatory apparatus was sized (per preregistered power plan) to detect large absolute differences (planned MDE ≈ 0.15). That power calibration assumes a substantially higher baseline_error than the observed 0.005. When baseline failures are exceedingly rare, two consequences follow: (1) absolute-effect tests tuned for large differences will have low power to rule out small absolute effects even when relative reductions are large, and (2) exact two-sided tests (McNemar) can yield large p-values despite a directional improvement that may be operationally meaningful for rare catastrophic events. Our paired bootstrap CI $[0.0, 0.015]$ shows the absolute paired improvement is small and imprecisely estimated under this N; the CI includes zero, so the preregistered absolute-difference confirmatory test does not reject the null at $\alpha=0.05$. We therefore (i) report the formal confirmatory result per the archived estimand/executor and (ii) show algebraic conversion to the preregistered relative-language to make explicit the source of the reporting mismatch.

Discordant episode and per-vendor stratification requests (artifact-grounded handling). Reviewers requested the explicit discordant episode identifier (the single baseline-invalid $\rightarrow$ guarded-valid pair) and a verbatim per-vendor stratified numeric table (baseline/guarded successes by preregistered strata). Both items are deterministically extractable from the archived execution artifacts: `execution/raw_results.jsonl` (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401b168402d) and archived in analysis/paired_contingency_table.csv (SHA-256 =

4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a) and the per-episode scoring JSONs under execution/scoring/ (full file-list manifest and SHA-256s are published in execution/execution_manifest.json). Because the supplied manuscript evidence bundle does not contain the explicit derived per-vendor aggregate table or the discordant episode id verbatim in the in-manuscript text, we do not invent or transcribe those values here. Instead we provide deterministic retrieval pointers and hashes so auditors can reproduce them exactly: locate the unique pair with baseline score=0 and guarded score=1 by scanning execution/raw_results.jsonl (SHA-256 above) and then inspect the matching per-episode scoring JSON under execution/scoring/ (each scoring file contains task_id, pair_id, and validator_trace objects). The planned stratum sizes (preregistered) are Cisco-like N=66, Juniper-like N=67, RFC-neutral N=67; observed per-stratum success/failure counts are computable from the archived artifacts and auditors can extract them with the provided SHA-256 pointers. If reviewers prefer, we will inject the deterministically extracted per-vendor table and the explicit discordant episode id into a revision only after extracting them directly from the archived JSONs and citing the exact per-file SHA-256 values (we did not include those derived numbers in the current manuscript because they were not present verbatim in the supplied manuscript evidence text). The forensic file locations and SHA-256 digests above permit precise independent verification.

Representative diagnostic episode(s). The single discordant improvement ($n_{10} = 1$) is recorded as a unique episode in execution/raw_results.jsonl and the per-episode scoring JSON set under execution/scoring/; the full hash manifest and episode-level traces are available for auditors (execution/execution_manifest.json listing all per-file SHA-256 values). Forensic auditors can deterministically locate the discordant episode and inspect its validator_trace (validation_before/after) using the archived raw_results and scoring JSONs. Representative per-episode scoring JSONs for examples appear under execution/scoring/ (see artifact_examples in the evidence bundle) but the discordant episode itself is not among the six illustrative examples supplied in the manuscript bundle.

V. DISCUSSION

Expanded discussion (artifact-grounded technical detail and diagnostics).

1) Reconciling estimand choice, hypothesis wording, and practical decision rules. The practical difference between a relative-error-reduction hypothesis (" $\geq 50\%$ relative reduction in actionable-error rate") and the preregistered absolute paired estimand (guarded - baseline success-rate difference) is numerical and decision-relevant in low-prevalence regimes. The archived estimator used for inference and for the preregistered decision rule is the absolute paired difference (paired_success_rate_difference_guarded_minus_baseline),

658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d0414dbf) oratory and should be examined on larger samples and analysis/condition_summary.csv (SHA-256 = 86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed00f728a68a) or deliberately higher baseline-error strata. The per-episode trace objects are the forensic basis for such classification and are archived per-episode under execution/scoring/ (SHA-256 pointers in Disclosure).

Analysts and designers should therefore interpret confirmatory outcomes in terms of the absolute paired difference and its CI rather than raw relative percentages when baseline_error is small. We emphasize that the observed 1→0 improvement yields an absolute paired change of 0.005; algebraically this corresponds to a 50% relative reduction only if the baseline error equals 0.01; for baseline_error=0.005 the equivalent absolute reduction to satisfy a 50% relative reduction would be 0.0025 — a quantity outside the scope of the preregistered power plan sized for large absolute effects. This arithmetic mapping is deterministic from the archived counts and is the correct reconciliation between the hypothesis wording and the estimator used.

2) Practical audit workflow for the discordant episode and per-vendor aggregation. The execution repository contains every per-episode trace required to reproduce the contingency cell and to inspect the validator’s before/after ASTs: execution/raw_results.jsonl (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af40bcb08402) execution/task_manifest.jsonl (SHA-256 = 4b28215e8a611feda50f6284058083b9ffc2f5881f557081ce9e9f89061cd8a) and the per-episode scoring JSONs under execution/scoring/ (full hash manifest under execution/execution_manifest.json). A reproducible forensic procedure is: (a) locate the single discordant pair in analysis/paired_contingency_table.csv (n_10 = 1), (b) identify its pair_id in execution/raw_results.jsonl, (c) open execution/scoring/<episode>.json to inspect validation_before and validation_after objects (ASTs, parsed_command_count, violation lists), and (d) verify provider-call provenance via execution/provider_calls/*. The repository-anchored SHA-256 values in the Disclosure allow auditors to verify file integrity before inspection. We implemented these steps in the internal analysis executor and recorded the actions in analysis/analysis_log.jsonl (SHA-256 = dbc0bd6b29e8624a2f72a35b0073a6ee7644daa85db4a7948a9fa75e27f6c63d3). Because the artifact set is canonical, third-party auditors can deterministically reconstruct the same episode-level narrative without ambiguity.

3) Validator coverage diagnostics and residual error taxonomy. The deterministic validator produces structured validation_before and validation_after objects that include parsed_command_count, required_command_count, normalized_configuration, and explicit violation_codes. These fields enable reproducible classification of residual errors into syntactic/parse (validator cannot parse or vendor-syntax fails), workflow/sequencing (intent-ordering, make-before-break violations), and semantic/policy mismatches (simulator-detected reachability or group constraints). In this run the guarded condition produced zero residual actionable errors (guarded_successes = 200). Because the residual count is zero, the preregistered H2 contingency test (residual error-type distribution comparison) was untestable as a confirmatory comparison; any discussion of residual error-type predomi-

4) Adapter-contract semantics and stage accounting implications. The adapter enforced shared_initial_generation semantics (single shared candidate across conditions) and a strict repair budget (maximum_repair_calls_per_task = 1). Provider-call accounting in analysis/deterministic_reconciliation.json (SHA-256 = 8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510e2593189) shows hosted_model_call_event_count = 201 and stage_counts = {"shared_initial_generation": 200, "repair": 1}, matching model_calls_used = 201. This accounting explains why the guarded condition yielded zero additional shared initial provider-calls: the pipeline reused the same initial output and exercised the repair stage only when invoked. Designers should note that adapter budgeting interacts nonlinearly with validator coverage: with a generous repair budget multiple repair attempts could correct more baseline errors but also expose potential failure modes (LLM collusion with validator hints or overfitting to validator messages). The archived execution/execution_manifest.json and deterministic_reconciliation.json documents each call and is the source of truth for auditing these interactions.

5) Power, estimand selection for rare-failure regimes, and design recommendations. This confirmatory run demonstrates a common practical pitfall: powering a trial for large absolute effects (e.g., $\Delta \approx 0.15$) while the true baseline_error is rare (here 0.005). When baseline failure prevalence is small relative to target absolute reductions, paired binary estimands produce very low event counts and wide relative uncertainty for requested relative-reduction claims. Remedies: (a) increase N; (b) oversample high-difficulty strata where baseline_error is larger (the task generator encodes difficulty per task_payload.difficulty and auditors can filter by it deterministically); (c) adopt weighted estimands that upweight rare catastrophic-error types or cost-weighted outcomes; or (d) pre-specify hierarchical or Bayesian estimands that borrow strength across strata. All these alternatives are compatible with the existing forensic artifact model and can be simulated deterministically from the archived task templates and generator seeds in execution/task_manifest.jsonl.

6) Threats to construct and external validity tied to validator scope. The deterministic validator is necessarily an abstraction: it operationalizes a subset of vendor syntax, workflow policies, and semantic assertions available in the simulator. Residual unobserved semantic risks (e.g., vendor-specific side-effects outside the validator model) remain a concern and are explicitly bounded by the synthetic task bank’s policy language. Auditors should inspect validator coverage by sampling validator_trace.violation_codes across episodes to locate unmodeled policy types; those JSON fields are authoritative for coverage analyses.

7) Reproducibility practice and reviewer requests.

Reviewers requested inline per-vendor stratified counts and the explicit discordant episode identifier. Both are deterministically derivable from the archived artifacts; the repository contains the authoritative JSONs needed to extract them (execution/task_manifest.jsonl SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a execution/raw_results.jsonl SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0 execution/scoring/*.json SHA-256s listed in execution/execution_manifest.json). We preserve deterministic reproducibility by pointing reviewers to these exact artifact paths and SHA-256 digests; auditors can extract and verify per-vendor aggregates and the discordant-pair id without ambiguity. If reviewers require those numbers printed verbatim in-manuscript we will include them in a subsequent revision only after deterministic extraction (to avoid introducing errors in reporting), citing the extracted CSV with its SHA-256 digest inline.

8) Operational implication summary. Under the conservative adapter contract and the validator implementation used, the practical marginal benefit of adding a deterministic validator is tightly coupled to baseline_error prevalence and to the validator’s semantic coverage. For organizations with very low baseline LLM error rates, validator investment should be guided by the cost of a single failure and by targeted improvements in validator semantic coverage for catastrophic categories rather than by expecting large absolute proportion reductions. For higher baseline-error contexts, deterministic validators with broader simulator-backed assertions and richer repair budgets are more likely to produce material absolute gains; these configurations can be explored reproducibly from the archived task templates and generator seeds.

In sum, the discussion expands the artifacts-to-decision pipeline: arithmetic reconciliation of estimand vs hypothesis, concrete auditing steps to locate and inspect the discordant episode and per-vendor aggregates, validator-coverage diagnostics using per-episode JSON fields, and design recommendations for future confirmatory runs in rare-failure regimes. All diagnostic steps above reference specific archived files and SHA-256 digests in the Disclosure so auditors can reproduce every assertion deterministically.

VI. LIMITATIONS

- Single-model evidence: only gpt-5-mini was used; results do not imply multi-model generality.
- Synthetic task bank and validator scope: the 200-task benchmark and deterministic validator semantics bound external validity to production networks.
- Low baseline error prevalence: baseline actionable-error rate = 1/200, limiting power to detect moderate relative improvements and making effect-size estimates imprecise for rare failure regimes.
- Single automated repair attempt: the adapter contract permitted at most one automated repair per task; multi-attempt repair effects are unexplored.

- Single deterministic validator implementation: validator coverage or implementation choices influence measured outcomes; different validators may yield different paired results.
- Untestable preregistered secondary hypothesis (H2): because guarded residual failures = 0, the preregistered confirmatory contingency test comparing residual error-proportions could not be executed and any error-type comments are exploratory.
- Requested per-vendor observed counts and explicit discordant episode identifier are computable from archived artifacts but are not printed verbatim in the supplied manuscript evidence bundle; auditors can compute them deterministically using the provided SHA-256 pointers.

VII. CONCLUSION

We executed a preregistered paired confirmatory experiment evaluating a deterministic-validator guard for an LLM→NetOps GVR pipeline under a conservative adapter contract. On a synthetic 200-task bank using gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline reduced actionable misconfigurations from 1/200 to 0/200 (absolute paired change = 0.005). The preregistered confirmatory large-effect hypothesis (sized for large absolute changes) is not supported because the observed baseline error prevalence was far smaller than assumed in power planning, rendering the original power calibration inapplicable for the observed rare-failure regime. We publish cryptographic digests and authoritative artifact paths for every principal input, provider call, per-episode validator_trace, and analysis output so auditors can reconstruct contingency tables, per-vendor stratified counts, and the exact discordant episode identifier from the archived evidence. Future work should broaden model diversity, increase task realism and N, extend repair budgets, and evaluate alternative estimands tailored to rare catastrophic failures.

DISCLOSURE STATEMENT

Disclosure Statement (mandatory). Initial master prompt immutable reference: provenance/master_prompt.txt SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15 Preregistration: cnsmspec2026_gvr_v1_prereg_001 SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099eb7da6 Declared model and configuration: execution/model_configuration.json (requested_model = gpt-5-mini) SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82 Execution raw results and task manifest: execution/raw_results.jsonl SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0 execution/task_manifest.jsonl SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a Deterministic reconciliation and provider-call accounting: analysis/deterministic_reconciliation.json SHA-256 = 8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510e2593189 Paired contingency evidence: analysis/paired_contingency_table.csv SHA-256 =

658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d9414db0.

REFERENCES

Condition summary (success rates): anal- [1] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.228.5064>
ysis/condition_summary.csv SHA-256 = [2] <https://doi.org/10.1145/2342441.2342452>
86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed0b628a68a [3] <https://doi.org/10.1145/3656296>
Missingness summary: analy- [4] <https://doi.org/10.1109/mcom.001.2400368>
sis/missingness_summary.csv SHA-256 =

808b3c00ef1a906db9c3422947d9bb9997089bbebbf3c9ebc731353240265c1c.
Analysis executor inputs: analy-
sis/results.json (input results SHA-256 =

9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02).
Adapter and tooling: adapter_family = hosted_netops_gvr_v1;

deterministic validator = deterministic_netops_validator_v5;

maximum repair calls per task enforced = 1. Provider-

call accounting explicit: provider_call_audit shows

hosted_model_call_event_count = 201 and stage_counts

= {"shared_initial_generation": 200, "repair": 1}, which

explains model_calls_used = 201 = 200 shared_initial +

1 repair (see execution/model_configuration.json SHA-256

above and deterministic_reconciliation.json SHA-256 above).

Contingency-cell notation and CSV ordering: the archived

CSV analysis/paired_contingency_table.csv uses header

'guarded,baseline,count' (guarded first, baseline second); we

define n_11 as guarded=1 & baseline=1, n_10 as guarded=1 &

baseline=0 (guarded-only improvements), n_01 as guarded=0

& baseline=1 (baseline-only), and n_00 as guarded=0 &

baseline=0. Observed values in this run: n_11 = 199, n_10 = 1,

n_01 = 0, n_00 = 0; the single discordant pair is therefore an

improvement (baseline-invalid → guarded-valid). Human role

and autonomy boundary: a human Research Director selected

the broad topic family and initial constraints pre-lock. After

the autonomy lock the autonomous researcher performed

literature verification, study selection, preregistration,

code generation, execution, analysis, manuscript drafting,

autonomous internal reviews, and revision. No human manual

editing of the technical content, numeric results, or claims

occurred after the autonomy lock. All authoritative files

and hashes above are archived in the execution repository

referenced by execution/execution_manifest.json and are

the source of truth for the corrected contingency labeling,

provider-call accounting, and analysis outputs. Reviewer

requests unresolved in-manuscript (but computable by

auditors): (1) a verbatim per-vendor stratified numeric

table (Cisco-like / Juniper-like / RFC-neutral) and (2) the

explicit discordant episode identifier string. Both values are

derivable from execution/task_manifest.jsonl (SHA-256 =

4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a)

together with execution/raw_results.jsonl (SHA-256 =

9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02)

and per-episode scoring JSONs under execution/scoring/ (full

hash manifest in execution/execution_manifest.json); because

neither value appears verbatim in the supplied manuscript

evidence bundle text we provide these artifact pointers

rather than invent numeric summaries or episode ids in the

manuscript where those values are not present verbatim.